Use case analysis is used to identify & define all business processes that system must support

Use case:

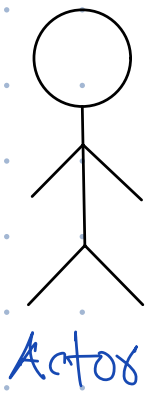
↳ An activity a system carried out, usually in response to a user request?

Use case

Actor:

- Primary actor

↳ They directly interact with the system



- Secondary actor:

↳ The system interacts with them.

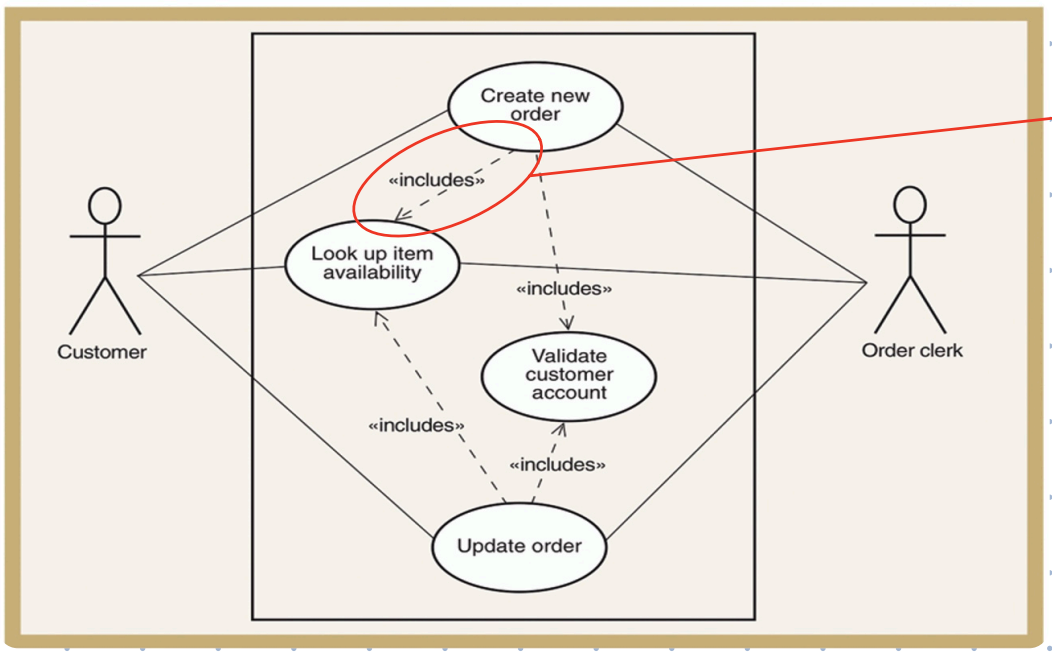
↳ They are never first to initiate contact.

↳ Use cases are enclosed in a rectangle & actors are outside of it, a line connects them showing actor participation called association line.

<<includes>> relationship

↳ When one use case requires services of a common subroutine

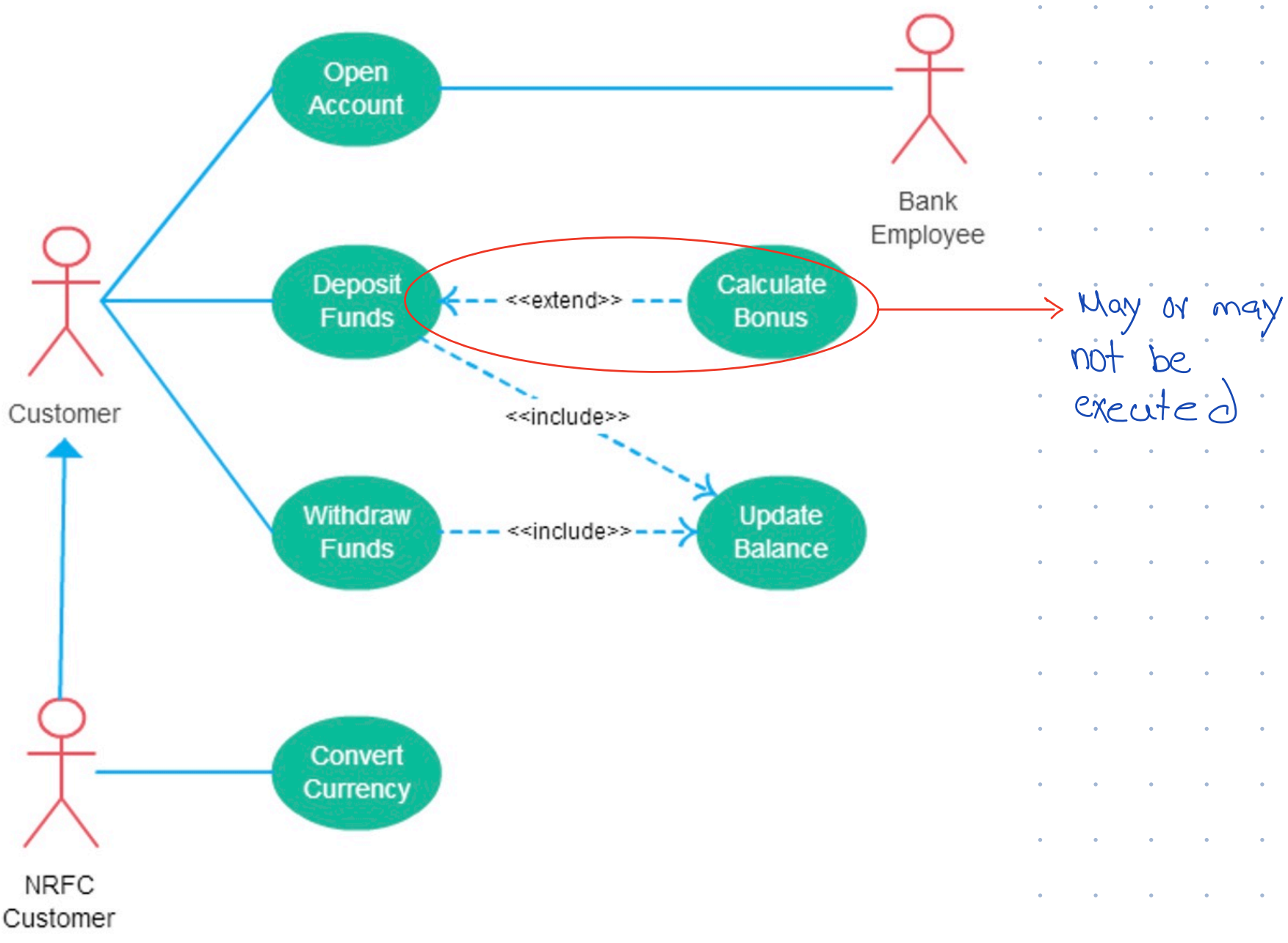
↳ Represents mandatory execution



→ when "create new order" is clicked, then "look up item availability" & rest are executed as well.

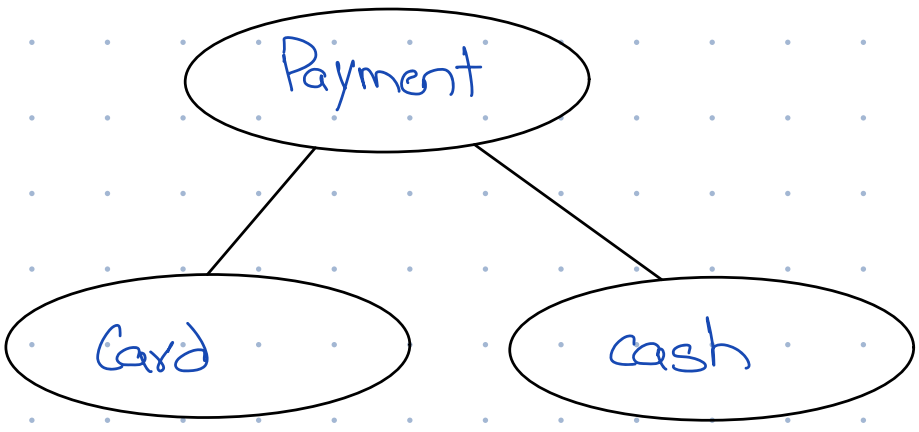
<<extends>> Relationship

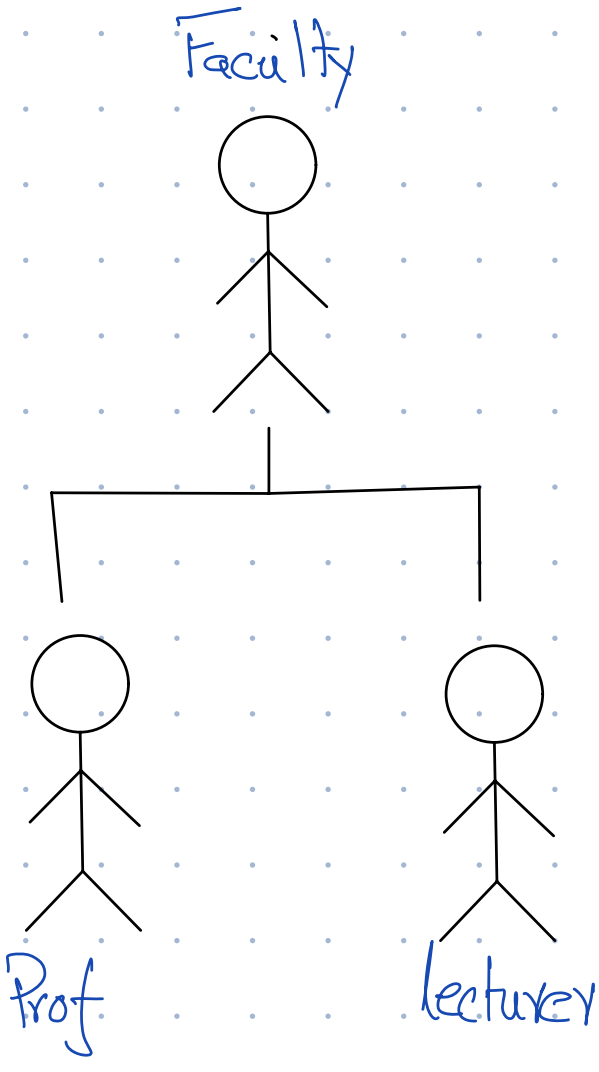
- Adds behaviour of base use case
- Represents optional execution upto user



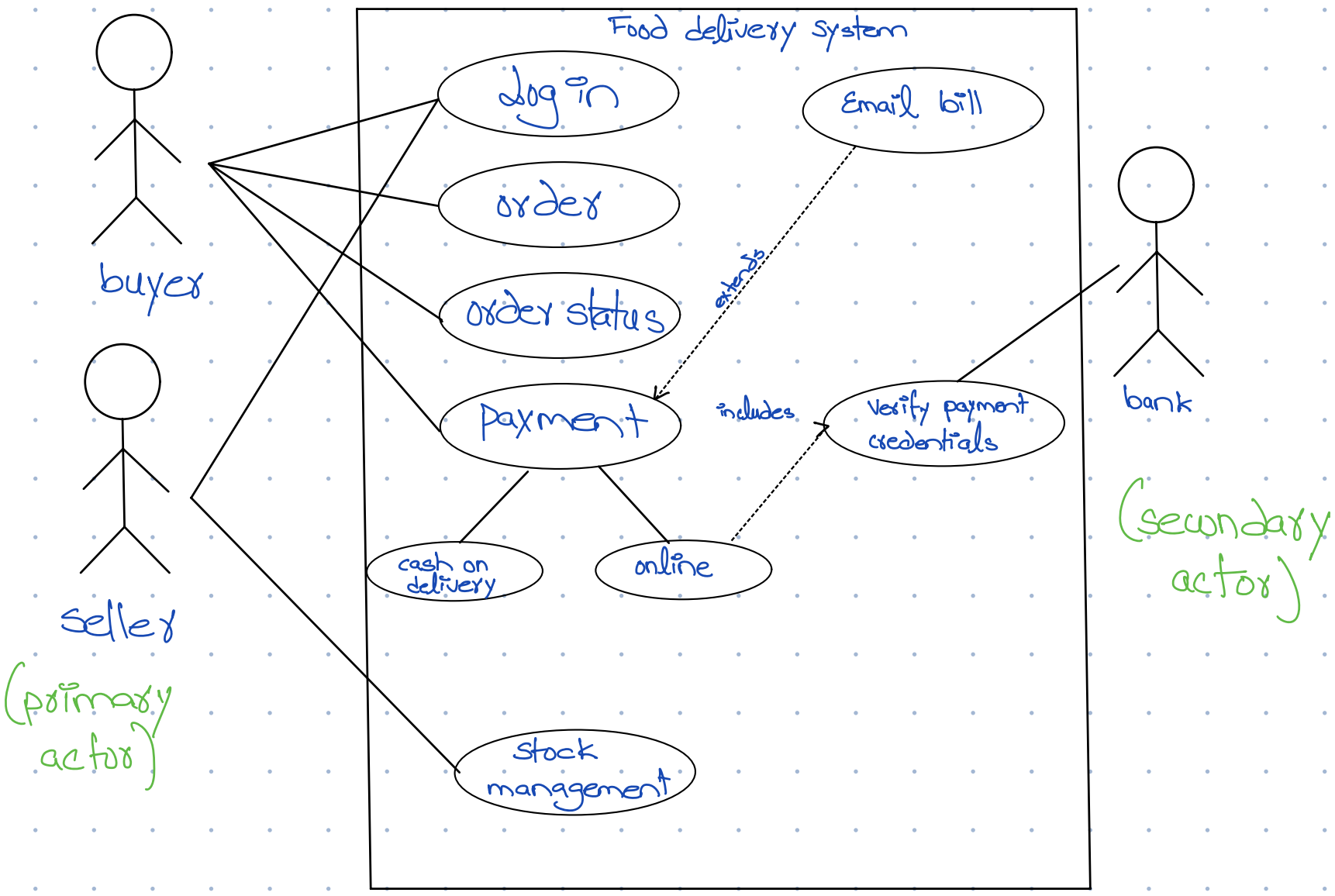
Generalization

- of actor or use cas





Example,



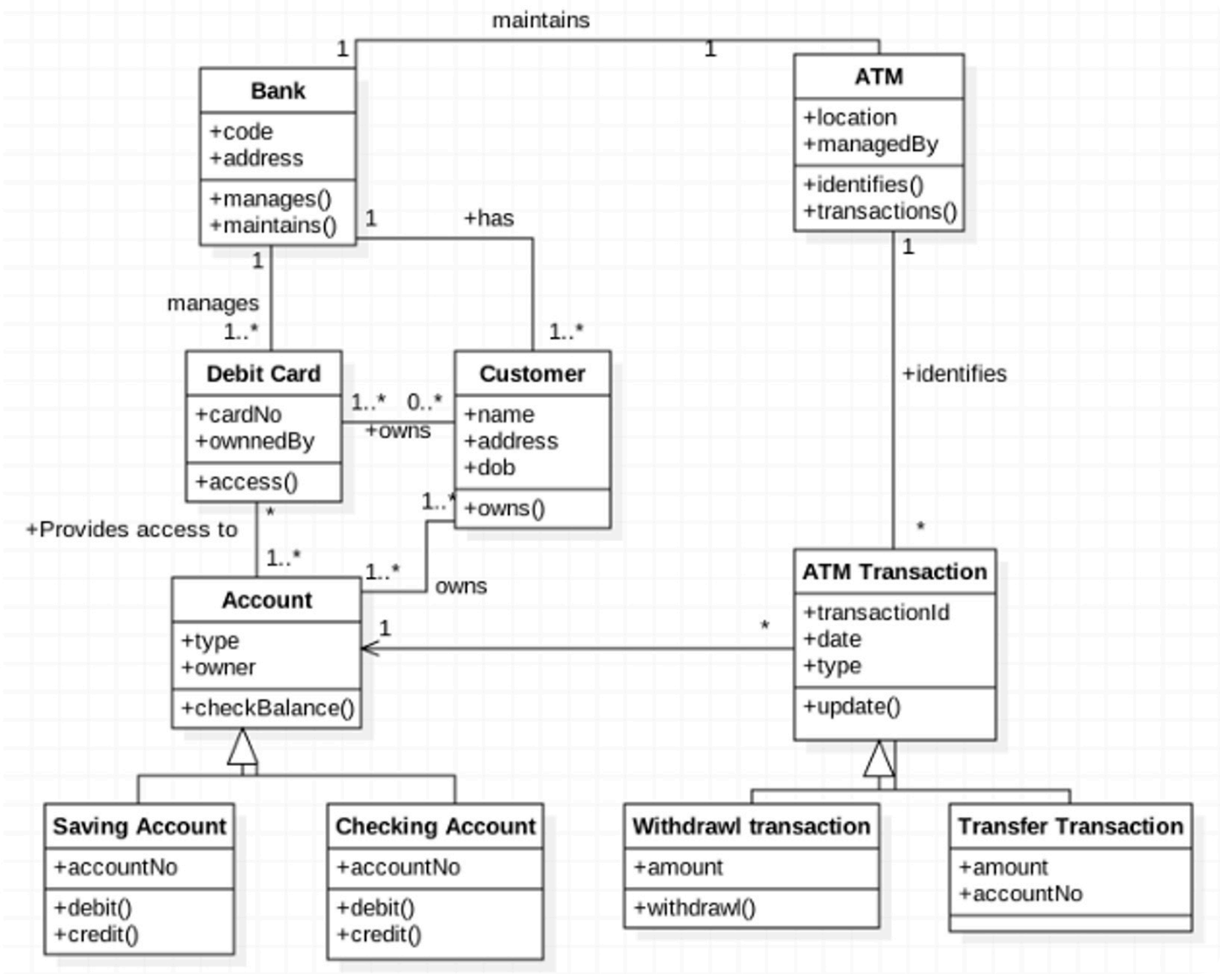
Use case description:

↳ Describes actor interacting with the system step-by-step to carry out activity/use case

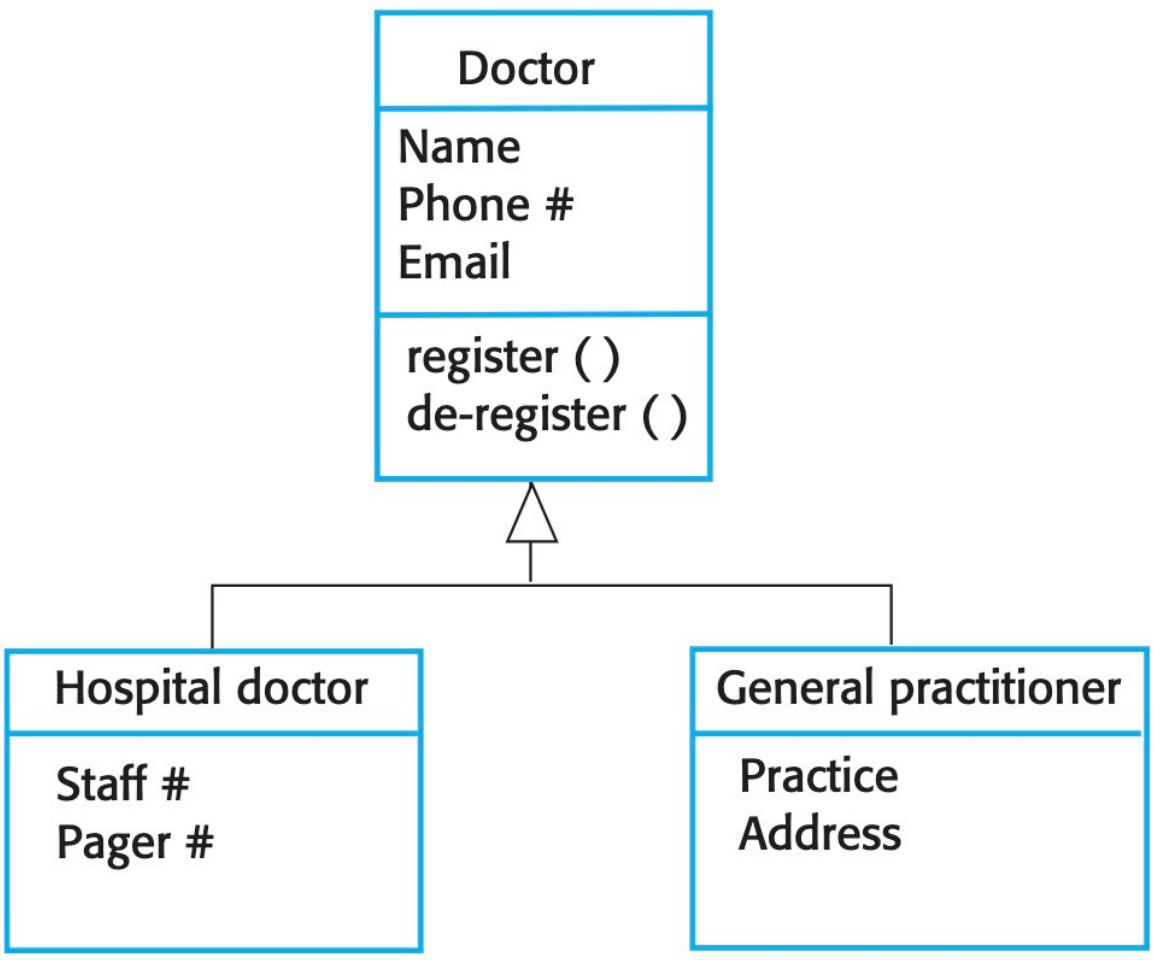
Use Case Section	Comment
Use Case Name	Start with a verb.
Scope	The system under design.
Level	"user-goal" or "subfunction"
Primary Actor	Calls on the system to deliver its services.
Stakeholders and Interests	Who cares about this use case, and what do they want?
Preconditions	What must be true on start, *and* worth telling the reader?
Success Guarantee (Post conditions)	What must be true on successful completion, *and* worth telling the reader.
Main Success Scenario	A typical, unconditional happy path scenario of success.
Extensions	Alternate scenarios of success or failure.
Special Requirements	Related non-functional requirements.

Class diagram

- Depicts classes & their inter-relationships
- Each class is represented by a rectangle sub-divided into three compartments
  1. Name
  2. Attributes
  3. Operations
- Modifiers are used to indicate visibility of attributes & operations
  - '+' : Public
  - '#' : Protected
  - '-' : Private



Generalization:



↳ Is-A relationship

Aggregation

↳ Where one class contains a collection of other classes.

↳ has-a relationship

↳ Each part/class can exist independently

Example,

A car has a wheel

Composition:

↳ When two or more entities are extremely reliant on each other.

↳ The composed object cannot exist without each other

↳ Represents part-of relation

Example,

A room is a part-of house

Data flow diagrams

- Describes the flow of data/information into & out of a system.
  - What does the system do to the data?

Main elements:

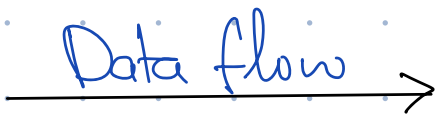
1. External entity:
   - People or organizations that send data into the system or recieve data from the system.

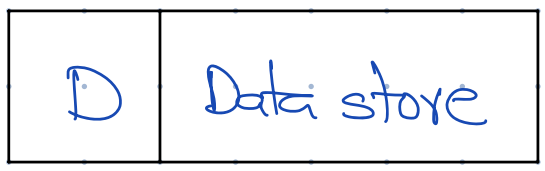
2. Process:
   - Models what happens to the data i.e transforms incoming data into outgoing data.

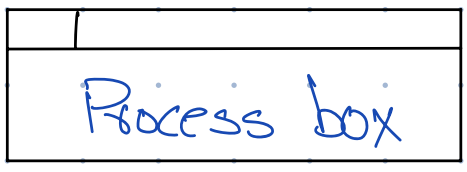
3. Data store:
   - Represents permanent data that is used by the system

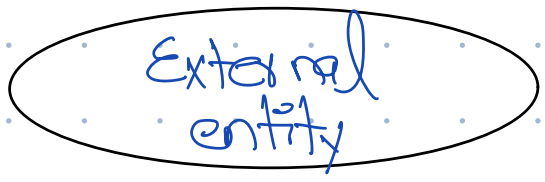
4. Data flow:
   - Models the actual flow of the data b/w the other elements.

Notation









Levelled DFDs:
- View system at different levels of detail

Level 0:

- Models one system as one process box which represents the scope of the system.
- Identifies external entities & related inputs & outputs

Level 1:

- Identifies major processes & data flows b/w them.
- Identifies data stores that are used by the major processes.
- Processes are numbered x, y, z ...

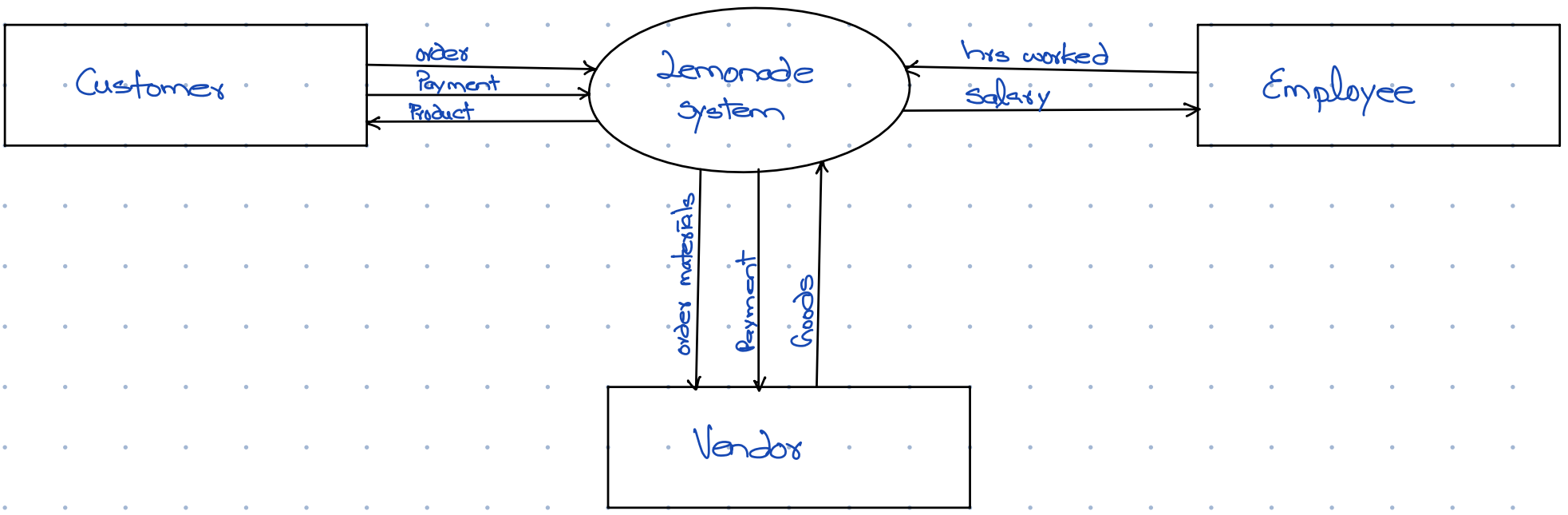
Level 2:

- Level 1 is expanded into more detail
- Each process in level 1 is decomposed to show its constituent processes.
- Processes are numbered x.1, x.2, y.1, y.2, y.3, ...
- Numbers are used to identify process & not order.
- Labels:
  - Process label:
    short description of what the process data
    e.g Price order
  - Data flow label:
    - Noun representing the data flowing
      e.g customer payment

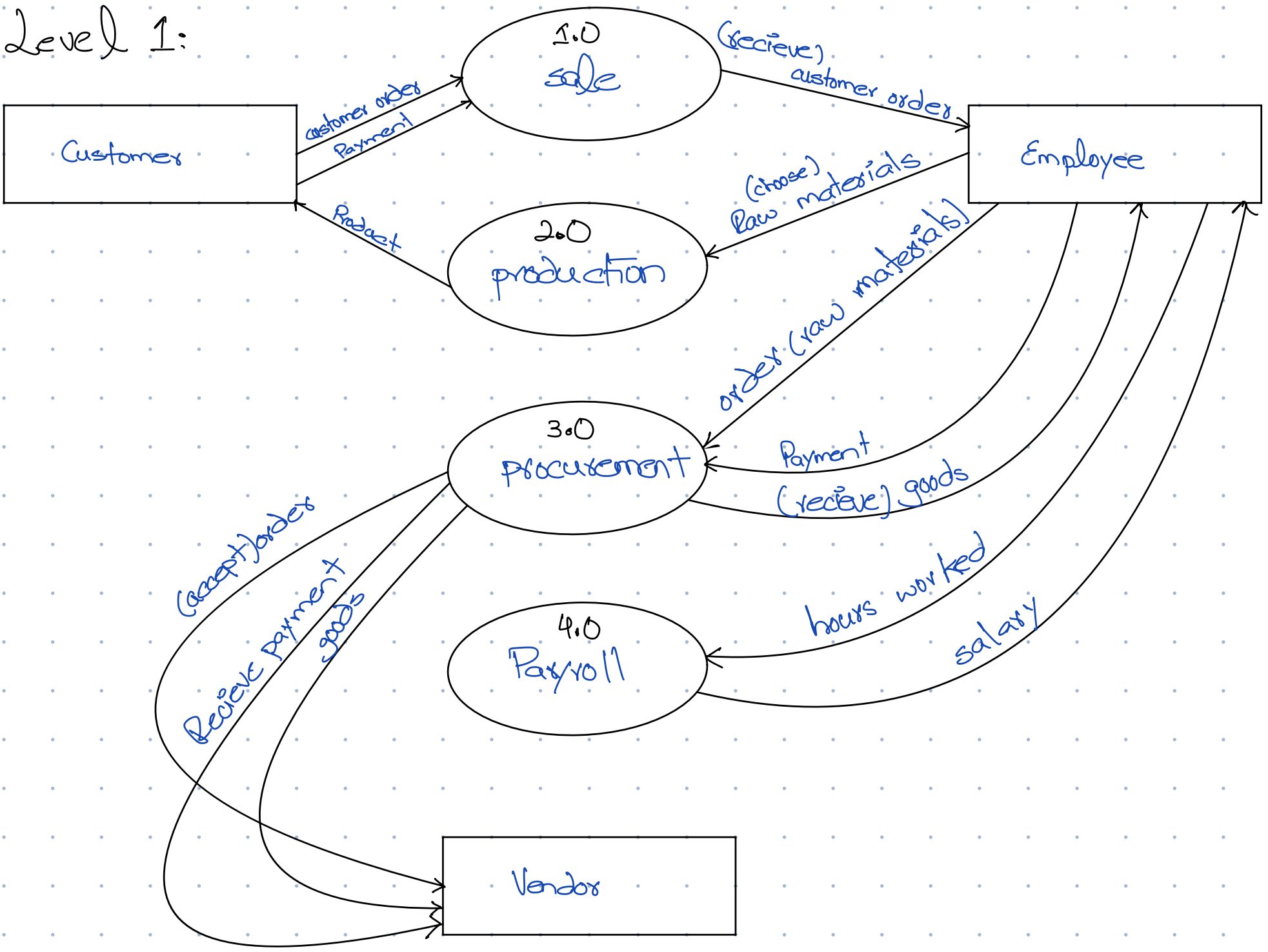
- Data store labels
  - Describe the type of data stored.

Example,

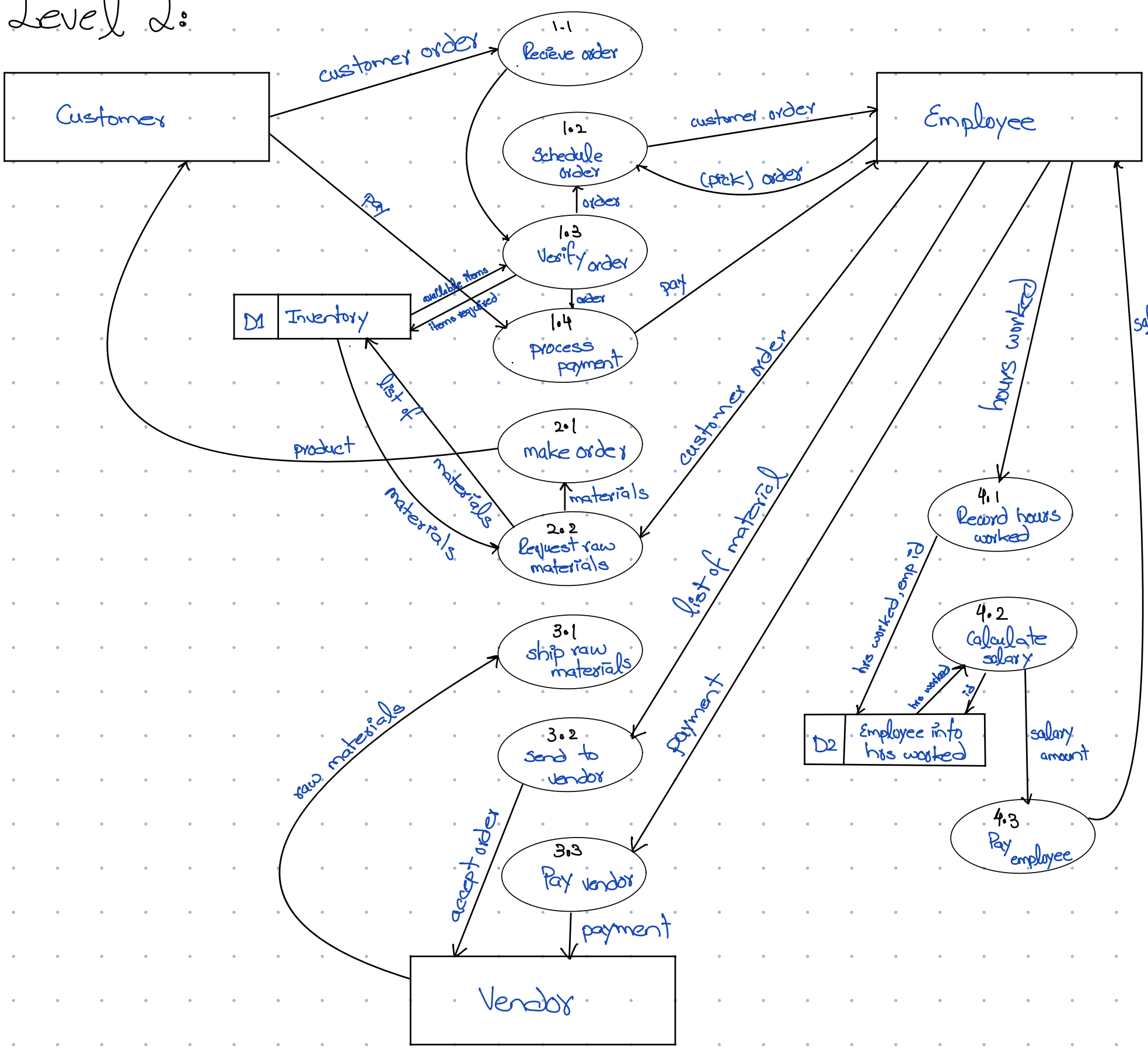
Level 0:



Level 1:



Level 2:



Rules to remember:

Data cannot flow directly between entities.

Entities (like Customer, Vendor, etc.) can only interact with processes or data stores, not with each other.

Entities must interact with processes.

External entities (e.g., Customer, Vendor) send data to processes and receive data from processes, not directly to data stores or other entities.

Data flows must represent the movement of data, not actions.

Data flows show information moving from one component to another, not actions or functions. They are **arrows**, not labels of processes.

Processes should represent actions that transform data.

Each process should perform a specific function, like validating an order or processing payment, and it should have at least one input and one output.

Processes interact with data stores, not entities.

Data stores are **repositories** where data is stored. Only processes should read from or write to data stores, never external entities.

Processes in Level 1 and Level 2 must be broken down logically.

Level 1 DFDs decompose high-level processes into sub-processes, and Level 2 should show more detail, breaking down processes into smaller, understandable steps.

Data stores only interact with processes, not entities or other data stores.

Data stores hold data and should only be accessed or modified by processes. Entities cannot read or write directly to data stores.

Each process should have both input and output data flows.

A process cannot exist without receiving data and outputting data to another process or entity. It’s not just a storage unit; it transforms or manipulates data.

Data stores should not have data flows to each other.

Data stores interact through processes. A data store doesn’t send or receive data directly to another data store.

External entities are not involved in data manipulation.

External entities send data to processes and receive data from processes but don’t perform any transformation or manipulation of the data themselves.

Following these rules ensures that your DFDs follow standard conventions and remain conceptually clear.